

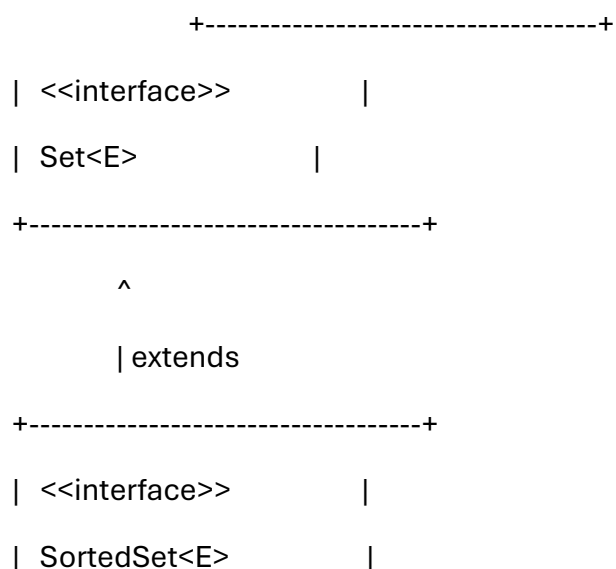
Aim:

To write a Java program that demonstrates the methods of the SortedSet interface, namely first(), last(), headSet(), tailSet(), subSet() and comparator(), using a TreeSet to show how elements are kept in sorted order and how portions of the set can be retrieved.

Algorithm:

1. Start.
2. Create a SortedSet of Integer using TreeSet, which keeps elements in sorted (natural) order.
3. Add the elements 30, 10, 50, 20 and 40 and display the set. It prints in sorted order.
4. Call first() to get the lowest element.
5. Call last() to get the highest element.
6. Call headSet(30) to get the elements strictly less than 30.
7. Call tailSet(30) to get the elements greater than or equal to 30.
8. Call subSet(20, 40) to get the elements from 20 (inclusive) to 40 (exclusive).
9. Call comparator() on the set. It returns null because natural ordering is used.
10. Create a second TreeSet with Collections.reverseOrder() and call comparator() on it. It returns a non-null comparator.
11. Stop.

UML Diagram:



```

+-----+
| + first(): E          |
| + last(): E          |
| + headSet(toElement: E): SortedSet<E>    |
| + tailSet(fromElement: E): SortedSet<E>  |
| + subSet(fromElement: E, toElement: E): SortedSet<E> |
| + comparator(): Comparator<? super E>    |

```

```

+-----+
      ^
      | extends

```

```

+-----+
| <<interface>>        |
| NavigableSet<E>       |

```

```

+-----+
      ^
      | implements

```

```

+-----+
| TreeSet<E>            |
+-----+

```

```

+-----+
| SortedSetDemo         |

```

```

+-----+
| + main(args: String[]): void |

```

```

+-----+
      |
      | uses
      v

```

SortedSet<E>

Program:

```
package Observations;

import java.util.Collections;
import java.util.SortedSet;
import java.util.TreeSet;

public class SortedSetInterface {

    public static void main(String[] args) {

        SortedSet<Integer> set = new TreeSet<>();

        set.add(30);

        set.add(10);

        set.add(50);

        set.add(20);

        set.add(40);


        System.out.println("Set: " + set);

        System.out.println("first(): " + set.first());

        System.out.println("last(): " + set.last());

        System.out.println("headSet(30): " + set.headSet(30));

        System.out.println("tailSet(30): " + set.tailSet(30));

        System.out.println("subSet(20, 40): " + set.subSet(20, 40));

        System.out.println("comparator() (natural ordering): " + set.comparator());


        SortedSet<Integer> reverseSet = new TreeSet<>(Collections.reverseOrder());

        System.out.println("comparator() (reverse order set) is not null: "

            + (reverseSet.comparator() != null));

    }
}
```

```
}
```

Output:

```
Set: [10, 20, 30, 40, 50]
first(): 10
last(): 50
headSet(30): [10, 20]
tailSet(30): [30, 40, 50]
subSet(20, 40): [20, 30]
comparator() (natural ordering): null
comparator() (reverse order set) is not null: true
PS C:\Users\Sahithi_pyla\Desktop\Sahithi_java>
```

Result:

The program was executed successfully. The TreeSet stored the elements in sorted order, and the range methods returned the expected portions of the set. comparator() returned null for natural ordering and a non-null comparator for the reverse-ordered set.